

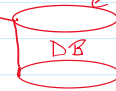
Prepared Statement

04 July 2026 11:47

```
Statement st = con.createStatement();
```

```
PreparedStatement pst = con.prepareStatement(query);
```

pre-compiled Query



compile the query

1 lakh time
 $\left\{ \begin{array}{l} \text{pst.executeQuery();} \rightarrow \text{just execute.} \\ \vdots \\ \text{pst.executeQuery();} \end{array} \right.$

Statement

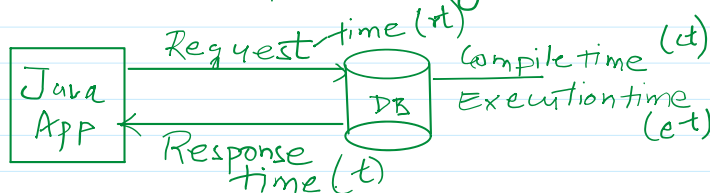
```
Statement st = con.createStatement();
```

```
st.executeUpdate(query);
```

```
st.executeUpdate(query);
```

```
st.executeUpdate(query);
```

1 crore time



For 1 query = $rt + ct + et + t$

= $1 + 1 + 1 + 1 = 4 \text{ ms}$

For 1000 query = $4 \times 1000 = 4000 \text{ ms}$

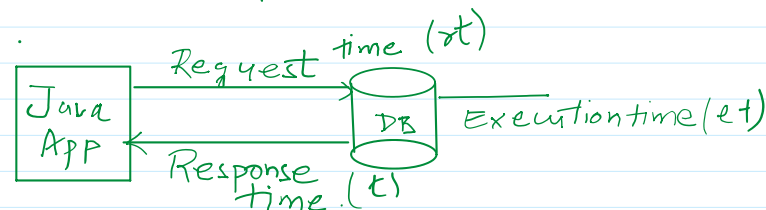
PreparedStatement

```
PreparedStatement ps = con.prepareStatement(query);
```

```
ps.executeUpdate();
```

1 crore time

```
ps.executeUpdate();
```



For 1 query = $rt + et + t$
 = $1 + 1 + 1 = 3 \text{ ms}$

For 1000 query = $3 \times 1000 = 3000 \text{ ms}$

Difference between Statement and PreparedStatement

Statement	PreparedStatement
At the time of creating Statement Object we are not required to provide any query.	At the time of creating PreparedStatement we have to provide SQL query compulsory and will send to the database and will be compiled.
Statement st = con.createStatement(); Hence the statement object is not associated with any query and we can use for multiple queries.	PreparedStatement ps = con.prepareStatement(query); Hence PreparedStatement is associated with only one query.
Whenever we are using execute method every time query will be compiled and	Whenever we are using execute method, query won't be compiled it will just be executed.

executed.	
Statement object can work only for static queries.	PreparedStatement object can work for both static and dynamic queries.
Relatively performance is low.	Relatively performance is high.
Best choice if we want to work with multiple queries.	Best choice if we want to work only One query but not required to execute multiple times.
There may be a chance of SQL injection attack.	There is no chance of SQL injection attack.
Inserting date and large objects (CLOB and BLOB) is difficult.	Inserting date and large objects. (CLOB and BLOB) is easy.

```

package MyJDBCOne;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;
import java.util.Scanner;
public class PreparedStatementDemo {
    public static void main(String[] args) {
        String dbUrl = "jdbc:oracle:thin:@//localhost:1521/FREE";
        String username = "sys as sysdba";
        String password = "Calcutta1983";
        Scanner sc = new Scanner(System.in);
        String query = "insert into employee values(?,?,?,?)";
        Connection con;
        try {
            con = DriverManager.getConnection(dbUrl, username, password);
            PreparedStatement ps = con.prepareStatement(query);
            int count = 0;
            while (true) {
                String empid, name, dep_id;
                float salary;
                System.out.println("Enter employee id: ");
                empid = sc.next();
                sc.nextLine();
                System.out.println("Enter name: ");
                name = sc.nextLine();
                System.out.println("Enter department id: ");
                dep_id = sc.next();
                System.out.println("Enter salary: \u20B9 ");
                salary = sc.nextFloat();
                ps.setString(1, empid);
                ps.setString(2, name);
                ps.setString(3, dep_id);
                ps.setFloat(4, salary);
                count += ps.executeUpdate();
                System.out.println("Record inserted successfully");
                System.out.println("Do you want to insert one more record, press y/Y: ");
                char option = sc.next().toLowerCase().charAt(0);
                if (option == 'y'){
                    continue;
                }
                else{
                    System.out.println("Number of records inserted: "+ count);
                    break;
                }
            }
        }
    }
}

```

```

    } catch (SQLException e) {
        e.printStackTrace();
    }
}
}

```

Interface PreparedStatement

All Superinterfaces:

AutoCloseable, Statement, Wrapper

All Known Subinterfaces:

CallableStatement

```

public interface PreparedStatement
extends Statement

```

An object that represents a precompiled SQL statement.

A SQL statement is precompiled and stored in a `PreparedStatement` object. This object can then be used to efficiently execute this statement multiple times.

Note: The setter methods (`setShort`, `setString`, and so on) for setting IN parameter values must specify types that are compatible with the defined SQL type of the input parameter. For instance, if the IN parameter has SQL type `INTEGER`, then the method `setInt` should be used.

If arbitrary parameter type conversions are required, the method `setObject` should be used with a target SQL type.

In the following example of setting a parameter, `con` represents an active connection:

```

BigDecimal sal = new BigDecimal("153833.00");
PreparedStatement pstmt = con.prepareStatement("UPDATE EMPLOYEES
        SET SALARY = ? WHERE ID = ?");

pstmt.setBigDecimal(1, sal);
pstmt.setInt(2, 110592);

```

Since:

1.1